

소스 변경 상세 — 무엇이 어떻게 바뀌었나

①번 문서의 **파일 14개**가 실제로 어떻게 바뀌는지 전·후로 보여 줍니다. 파일을 통째로 교체하실 거면 **6절**만 보셔도 되고, 손으로 고치거나 대조 검수를 하실 때 2~5절을 쓰십시오.

작성일 2026-09-08 (확정본) 기준 내부망 반입 zip(2026-09-07) 대비 선행 문서 ① 스키마 변경

검증 `mvn -o clean test` 62건 통과

1 왜 자바까지 바꾸나 — TypeHandler

MyBatis의 `resultMap` 은 자바 필드와 DB 컬럼을 잇는 번역표이고, `TypeHandler`는 그 번역표가 값을 어떻게 바꿀지 정하는 규칙입니다. 표준이 요구하는 값(문자열 시각·2자리 코드)과 자바가 쓰는 값이 다르므로, 변환을 세 클래스에 가두어 도메인·서비스·화면·테스트를 지켰습니다.

TypeHandler	자바가 쓰는 값	DB 에 들어가는 값
LocalDateTimeStringTypeHandler	LocalDateTime	'20260908103000'
ClassificationStatusTypeHandler	"UNCLASSIFIED"	'01'
InstitutionCategoryTypeHandler	"지방자치단체"	'01'

규칙 두 개만 지키면 됩니다

- resultMap** 과 `#{...}` 양쪽에 적는다. 두 코드 핸들러가 모두 자바 `String` 을 다뤄 자동 등록을 쓸 수 없습니다. 한쪽만 적으면 읽거나 쓰기 중 한 방향만 변환돼 조용히 어긋납니다.
- SQL** 에 직접 박은 값은 코드로 적는다. `TypeHandler` 는 파라미터와 결과에만 걸립니다. `SET` 분류 상태구분 = '04' 자리에 'DELETED' 를 적으면 오류 없이 0건이 바뀝니다.

2 신규 자바 6개 — 무엇을 하는 파일인가

파일	역할	고칠 일이 생기면
code/InstitutionCategory.java	기관영업분류코드 코드↔이름 5쌍. 화면 드롭다운 목록의 유일한 출처	분류를 늘리거나 이름을 바꿀 때 이 파일만
code/ClassificationStatus.java	분류상태 코드↔값 4쌍 + 상수	상태를 늘릴 때(도메인도 함께)
config/CodeTypeHandler.java	코드 변환 공통 뼈대(추상). 모르는 값이면 예외를 던진다	거의 없음
config/InstitutionCategoryTypeHandler.java	위 뼈대 + 기관영업분류코드 표	없음
config/ClassificationStatusTypeHandler.java	위 뼈대 + 분류상태 표	없음
config/LocalDateTimeStringTypeHandler.java	<code>yyyyMMddHHmmss</code> 변환. 읽을 때 <code>trim()</code> 한다	일시 형식이 바뀌면

모르는 값이 오면 조용히 넘기지 않습니다

```
ClassificationStatusTypeHandler: 코드로 옮길 수 없는 값이다 - '지자체'. 아는 값: [...]  
InstitutionCategoryTypeHandler: DB 에 모르는 코드가 들어 있다 - '99'. 아는 코드: [01, 02, ...]  
일시 형식이 YYYYMMDDHH24MISS 가 아니다 - '2026-09-08 10:30:00'
```

코드가 어긋나면 증상이 “저장은 됐는데 조회가 안 됨”처럼 엉뚱하게 나타나므로, **그 자리에서 예외로 멈추게** 했습니다. 위 메시지가 보이면 원인이 바로 드러납니다.

3 UploadedFileMapper.xml

3-1. resultMap 9행 전부 + TypeHandler 4곳

```
전 <id      property="id"           column="ID"/>
<result property="originalName"    column="ORIGINAL_NAME"/>
<result property="storedPath"      column="STORED_PATH"/>
<result property="year"            column="FILE_YEAR"/>
<result property="institutionName" column="INSTITUTION_NAME"/>
<result property="category"        column="CATEGORY"/>
<result property="status"          column="STATUS"/>
<result property="uploadedAt"      column="UPLOADED_AT"/>
<result property="classifiedAt"    column="CLASSIFIED_AT"/>
```

```
후 <id      property="id"           column="업로드파일일련번호"/>
<result property="institutionName" column="기관명"/>
<result property="category"        column="기관영업분류코드"
      typeHandler="com.kb.uploader.config.InstitutionCategoryTypeHandler"/>
<result property="year"            column="문서년"/>
<result property="status"          column="분류상태구분"
      typeHandler="com.kb.uploader.config.ClassificationStatusTypeHandler"/>
<result property="originalName"    column="원본파일명"/>
<result property="uploadedAt"      column="업로드일시"
      typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
<result property="classifiedAt"    column="분류일시"
      typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
<result property="storedPath"      column="저장경로"/>
```

3-2. <sql id="값목록"> 신설 — 파라미터 쪽 표기가 길어 묶었다

```
신설 <sql id="값목록">
    #{institutionName},
    #{category,typeHandler=com.kb.uploader.config.InstitutionCategoryTypeHandler},
    #{year},
    #{status,typeHandler=com.kb.uploader.config.ClassificationStatusTypeHandler},
    #{originalName},
    #{uploadedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
    #{storedPath}
</sql>
```

3-3. insert oracle — 내부망이 실제로 쓰는 쪽

```
전 SELECT UPLOADED_FILE_SEQ.NEXTVAL FROM DUAL
INSERT INTO UPLOADED_FILE
    (ID, ORIGINAL_NAME, STORED_PATH, FILE_YEAR, INSTITUTION_NAME, CATEGORY, STATUS, UPLOADED_AT)
VALUES
    (#{id}, #{originalName}, #{storedPath}, #{year}, #{institutionName}, #{category}, #{status}, #{uploadedAt})
```

```
후 SELECT SQ_KGI_AGENTFILETABLE.NEXTVAL FROM DUAL
INSERT INTO TSKGIAF01
    (업로드파일일련번호, 기관명, 기관영업분류코드, 문서년, 분류상태구분, 원본파일명, 업로드일시, 저장경로)
VALUES
    (#{id}, <include refid="값목록"/>)
```

⚠ 시퀀스 이름도 바뀝니다. 여기가 틀리면 ORA-02289 로 업로드만 실패합니다.

3-4. update

```
전 UPDATE UPLOADED_FILE
SET STORED_PATH = #{storedPath}, INSTITUTION_NAME = #{institutionName},
    FILE_YEAR = #{year}, CATEGORY = #{category},
    STATUS = #{status}, CLASSIFIED_AT = #{classifiedAt}
WHERE ID = #{id}
```

```
후 UPDATE TSKGIAF01
SET 저장경로 = #{storedPath},
    기관명 = #{institutionName},
    문서년 = #{year},
    기관영업분류코드 = #{category,typeHandler=...InstitutionCategoryTypeHandler},
    분류상태구분 = #{status,typeHandler=...ClassificationStatusTypeHandler},
    분류일시 = #{classifiedAt,typeHandler=...LocalDateTimeStringTypeHandler}
WHERE 업로드파일일련번호 = #{id}
```

3-5. softDeleteById · rejectById 값을 코드로

```
전 UPDATE UPLOADED_FILE SET STATUS = 'DELETED' WHERE ID = #{id}
UPDATE UPLOADED_FILE SET STATUS = 'REJECTED' WHERE ID = #{id}
```

```
후 UPDATE TSKGIAF01 SET 분류상태구분 = '04' WHERE 업로드파일일련번호 = #{id}
UPDATE TSKGIAF01 SET 분류상태구분 = '03' WHERE 업로드파일일련번호 = #{id}
```

⚠ **여기가 이번 변경에서 가장 놓치기 쉬운 자리입니다.** 리터럴은 TypeHandler 를 타지 않아 'DELETED' 를 그대로 두면 오류 없이 아무 행동 안 바꿉니다.

3-6. findByStatus 정렬 신설 + 파라미터 변환

```
전 SELECT * FROM UPLOADED_FILE WHERE STATUS = #{status}
```

```
후 SELECT * FROM TSKGIAF01
WHERE 분류상태구분 = #{status,typeHandler=...ClassificationStatusTypeHandler}
ORDER BY 업로드파일일련번호 DESC
```

종전에는 ORDER BY 가 없었습니다. 목록 순서를 DB 에 맡긴 셈이라 삽입순처럼 보였을 뿐 보장이 아니었고, 인덱스가 붙는 순간 순서가 바뀝니다. 이 문장은 앱에서 가장 많이 쓰입니다 — 파일처리상태 화면·분류완료 목록 화면·재분류 잡·InstitutionService 두 곳, 모두 여섯 자리.

3-7. findRecent 정렬 기준을 PK 로

```
전 SELECT * FROM (
    SELECT * FROM UPLOADED_FILE ORDER BY UPLOADED_AT DESC
) WHERE ROWNUM <= #{limit}
```

```
후 SELECT * FROM (
    SELECT * FROM TSKGIAF01 ORDER BY 업로드파일일련번호 DESC
) WHERE ROWNUM <= #{limit}
```

업로드일시가 문자열 초 단위라 같은 초에 여러 건이면 순서가 뒤섞입니다. PK 는 그 경우에도 정확하고, PK 인덱스가 정렬을 그대로 처리합니다.

```
전 SELECT COUNT(*) FROM UPLOADED_FILE
SELECT COUNT(*) FROM UPLOADED_FILE WHERE STATUS = #{status}
... WHERE INSTITUTION_NAME = '알수없음' AND STATUS = 'CLASSIFIED' ORDER BY UPLOADED_AT DESC
... WHERE INSTITUTION_NAME LIKE '%' || #{keyword} || '%' ORDER BY UPLOADED_AT DESC
... AND (#{year} IS NULL OR FILE_YEAR = #{year})
... AND (#{keyword} IS NULL OR ORIGINAL_NAME LIKE '%' || #{keyword} || '%')
```

```
후 SELECT COUNT(*) FROM TSKGIAF01
SELECT COUNT(*) FROM TSKGIAF01 WHERE 분류상태구분 = #{status,typeHandler=...}
... WHERE 기관명 = '알수없음' AND 분류상태구분 = '02' ORDER BY 업로드일시 DESC
... WHERE 기관명 LIKE '%' || #{keyword} || '%' ORDER BY 업로드일시 DESC
... AND (#{year} IS NULL OR 문서년 = #{year})
... AND (#{keyword} IS NULL OR 원본파일명 LIKE '%' || #{keyword} || '%')
```

검색·페이징은 일부러 업로드일시 정렬을 유지했습니다

인덱스 (기관명, 업로드일시) 의 두 번째 열이 업로드일시라, 기관 입력이 드롭다운(정확 일치)이 되는 날 정렬까지 인덱스가 받게 하려는 것입니다. 여기를 일련번호로 바꾸면 그 이점이 사라집니다.

4 InstitutionMapper.xml

4-1. resultMap

```
전 <id      property="id"          column="ID"/>
<result property="name"          column="NAME"/>
<result property="category"      column="CATEGORY"/>
<result property="modifiedAt"    column="MODIFIED_AT"/>
```

```
후 <id      property="id"          column="기관일련번호"/>
<result property="name"          column="기관명"/>
<result property="category"      column="기관영업분류코드"
      typeHandler="com.kb.uploader.config.InstitutionCategoryTypeHandler"/>
<result property="modifiedAt"    column="수정일시"
      typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
```

4-2. 나머지 7문장

```
전 SELECT INSTITUTION_SEQ.NEXTVAL FROM DUAL
INSERT INTO INSTITUTION (ID, NAME, CATEGORY, MODIFIED_AT) VALUES ({id}, {name}, {category}, {modifiedAt})
INSERT INTO INSTITUTION (NAME, CATEGORY, MODIFIED_AT)                                -- mysql
UPDATE INSTITUTION SET NAME = {name}, CATEGORY = {category}, MODIFIED_AT = {modifiedAt} WHERE ID = {id}
DELETE FROM INSTITUTION WHERE ID = {id}
DELETE FROM INSTITUTION
SELECT * FROM INSTITUTION WHERE NAME = {name}
SELECT * FROM INSTITUTION ORDER BY NAME ASC
```

```
후 SELECT SQ_KGI_AGENTGIGANTABLE.NEXTVAL FROM DUAL
INSERT INTO TSKGIAF02 (기관일련번호, 기관명, 기관영업분류코드, 수정일시) VALUES ({id}, <include refid="값목록"/>)
INSERT INTO TSKGIAF02 (기관명, 기관영업분류코드, 수정일시)                                -- mysql
UPDATE TSKGIAF02 SET 기관명 = {name}, 기관영업분류코드 = {category,typeHandler=...},
                    수정일시 = {modifiedAt,typeHandler=...} WHERE 기관일련번호 = {id}
DELETE FROM TSKGIAF02 WHERE 기관일련번호 = {id}
DELETE FROM TSKGIAF02
SELECT * FROM TSKGIAF02 WHERE 기관명 = {name}
SELECT * FROM TSKGIAF02 ORDER BY 기관명 ASC
```

`findByName` 이 이 앱에서 가장 중요한 문장입니다

분류가 여기서 결정됩니다 — 업로드된 파일의 기관명으로 이 문장을 돌려 못 찾으면 파일이 미분류로 남습니다. 틀리면 오류 없이 “분류가 하나도 안 되는” 증상으로만 나타납니다.

TSKGIAF02 에 컬럼 4개가 늘었지만 매퍼는 바뀌지 않습니다

협약기관상태구분 · 입찰공고상태 · 입찰공고시작 · 입찰공고마감 이 신설됐으나 애플리케이션이 읽지도 쓰지도 않습니다. resultMap 은 모르는 컬럼을 그냥 무시하고 INSERT 는 컬럼을 명시하므로 NULL 로 남습니다 — 이 절과 4절에 추가로 손댈 것이 없습니다.

5 컨트롤러 3개 + 화면 1개

카테고리 목록이 세 곳에 복제돼 있었고 값이 옛 4종이었습니다. 새 5종과 다르므로 그대로 두면 `ExceptionHandler` 가 기관 등록을 거부합니다.

`KGI12000$FileStatusList.java` · `KGI12400$ClassifiedFileList.java`

```
전 private static final List<String> ALL_CATEGORIES =  
    Arrays.asList("지자체", "대학교", "대학병원", "공공기관");  
...  
model.addAttribute("categories", ALL_CATEGORIES);
```

```
후 import com.kb.uploader.code.InstitutionCategory;  
...  
model.addAttribute("categories", InstitutionCategory.names());
```

상수 선언과 `import java.util.Arrays;` 는 지웁니다.

`KGI13000$InstitutionList.java` 화면에 목록을 넘기도록 추가

```
추가 import com.kb.uploader.code.InstitutionCategory;  
...  
model.addAttribute("categories", InstitutionCategory.names());
```

`templates/institution.html`

```
전 <option value="지자체">지자체</option>  
<option value="대학교">대학교</option>  
<option value="대학병원">대학병원</option>  
<option value="공공기관">공공기관</option>
```

```
후 <option th:each="cat : ${categories}" th:value="${cat}" th:text="${cat}"></option>
```

다른 화면(`upload.html` · `file-status.html` · `classified-rework.html`)은 이미 `${categories}` 를 쓰고 있어 손댈 것이 없습니다.

6 내부망에서 적용하는 법

방법 ㉓ — 파일 교체 권장

외부망 리포	내부망 WTP 폴더
uploader/src/main/java/com/kb/uploader/**/*.java	src/com/kb/uploader/**/*.java
uploader/src/main/resources/mapper/*.xml	src/mapper/*.xml
uploader/src/main/resources/schema-*.sql	src/schema-*.sql
uploader/src/main/resources/templates/institution.html	src/templates/institution.html

신규 폴더 `src/com/kb/uploader/code/` 가 생깁니다. 교체 후 **Project** → **Clean** → **Publish**.

방법 ㉔ — 매퍼를 손으로 고칠 때의 치환 순서

순서를 지키지 않으면 XML 이 깨집니다

INSTITUTION → TSKGIAF02 를 먼저 돌리면 INSTITUTION_NAME · INSTITUTION_SEQ 까지 망가집니다.
ID 는 databaseId · keyProperty 같은 속성 이름에도 들어 있어 무차별 치환하면 문서가 통째로 깨 집니다.

#	찾을 것	바꿀 것	주의
1	UPLOADED_FILE_SEQ	SQ_KGI_AGENTFILETABLE	가장 길어서 먼저
2	INSTITUTION_SEQ	SQ_KGI_AGENTGIGANTABLE	—
3	INSTITUTION_NAME	기관명	INSTITUTION 보다 먼저
4	ORIGINAL_NAME	원본파일명	—
5	CLASSIFIED_AT	분류일시	—
6	MODIFIED_AT	수정일시	—
7	UPLOADED_AT	업로드일시	—
8	STORED_PATH	저장경로	—
9	FILE_YEAR	문서년	—
10	UPLOADED_FILE	TSKGIAF01	1번을 먼저 했어야 안전
11	INSTITUTION	TSKGIAF02	2·3번을 먼저 했어야 안전
12	CATEGORY	기관영업분류코드	한 번만
13	STATUS	분류상태구분	한 번만
14	NAME	기관명	⚠ InstitutionMapper 에서만
15	ID	업로드파일일련번호 / 기관일련번호	🔴 일괄 치환 금지 — 한 곳씩 눈으로
16	그다음 TypeHandler 를 손으로 넣는다(3-1·3-2·4-1) — 치환으로는 안 됩니다		
17	그다음 리터럴을 코드로(3-5의 '04' · '03', 3-8의 '02')		

16·17번 때문에 ㉔는 권하지 않습니다. 파일 교체가 훨씬 안전합니다.

7 적용 후 확인

확인	방법	정상
컴파일	Project → Clean 후 Problems 뷰	오류 0건. code 패키지가 보임
배포	WEB-INF/classes/mapper/*.xml 열어보기	TSKGIAF01 이 보임
기동	서버 로그	TypeHandler 예외 없음
드롭다운	/uploader/institutions	분류 선택지가 5종(지방자치단체·공공기관·대학교·병원·법원)
코드 저장	테이블 생성 후 기관 1건 등록 → DB 조회	기관영업분류코드 에 '01' 같은 코드
일시 저장	파일 1건 업로드 → DB 조회	업로드일시 가 20260908103000 14자

② uploader 소스 변경 상세 · 2026-09-08 확정본 · 선행 문서: 내부망작업지시_2026-09-08_1_스키마변경.pdf